

Aula 07 – Estruturas de dados em Python 3

Parte 1 – Revisão Teórica

Tipos de variáveis em Python

Como na maioria das linguagens temos o conceito de variáveis e tipos de dados e apesar da linguagem ser tipada dinamicamente, ela é **fortemente tipada** como veremos em breve.

Tipo	Seção	Exemplos
bool		True ou False
int	Números	3
float	Números	3.3
str	Strings	'João da Silva' ou "João da Silva"
list	Listas	[1, 2, 'ab']
dict	Dicionários	{'nome': 'João da Silva', 'idade': 21}
NoneType		None

Tabela 1. Tipos básicos de dados em Python

Estrutura de dados é qualquer meio utilizado para armazenar e recuperar dados

Principais estruturas

- 1) Lista
- 2) Tupla
- 3) dicionário
- 4) Array/Matriz/vetor
- 5) Set(grupo)
- 6) Pilha
- 7) Arvore

Sequências

Sequências são estruturas de dados que possuem algumas propriedades em comum. Essencialmente, elas são ordenadas e podem ser indexadas.

A função **range()** vista anteriormente é na verdade uma sequência. Falaremos agora de outras das principais sequências em Python.

a) Variáveis do tipo Vetor

Em outras linguagens de programação, listas são chamadas de **vetores** e possuem restrições **que Python não impõe**:

- ✓ Em Python, os valores de uma lista podem ser de qualquer tipo
- ✓ Em outras linguagens, os valores precisam ser do mesmo tipo
- ✓ Em Python
 - notas = [10, 5, 6.7, 2, 7.5]

VETORES []

Alunos que estudaram linguagens como o C/C++, Java, Pascal as vezes imaginam o tipo lista como sendo um vetor, porém, conceitualmente, a única relação entre ambas as estruturas é que as 2 contêm um conjunto de elementos indexados.

- 1) Vetores possuem uma quantidade limitada de elementos
- 2) TODOS os elementos devem ser do mesmo tipo
- 3) Não permitem slice/fatiamento
- 4) Não possuem índices negativos,

Em computação é comum manipular conjunto de valores.

- ✓ Exemplos: notas, preços, produtos, nomes.
- ✓ Para manipular um conjunto de dados de um mesmo tipo (inteiro, real, string) há uma estrutura de dados denominada de **vetor ou uma lista homogênea**
- ✓ Vetores são “agregados **homogêneos unidimensionais**” ou “**variáveis compostas unidimensionais**”
- ✓ Permitem agrupar um conjunto de valores de um mesmo tipo em uma única variável

Exemplo: Um vetor para armazenar 10 valores numéricos inteiros

	vetor									
Índice →	0	1	2	3	4	5	6	7	8	9
Conteúdo →			50							

Vetores em Python

- ✓ Python possui uma estrutura similar a vetores denominada listas.
- ✓ Lista é um conjunto ordenado de valores, onde cada valor é identificado por um índice ;
- ✓ Os valores na lista são denominados elementos.
- ✓ Podemos denominar uma lista de agregado homogêneo unidimensional se todos os elementos são do mesmo tipo.

Exemplo Vetor 01

Para acessar (ler ou escrever) uma posição do vetor, basta informar a posição entre colchetes

```
notas = [8.0, 5.5, 1.5]
```

```
media = (notas[0] + notas[1] + notas[2]) / 3
```

```
notas = [8.0, 5.5, 1.5]
for i in range(3):
    print("Valor do elemento da posição " + str(i), " = " , notas[i])
```

Listagem 1

```
notas = [8.0, 5.5, 1.5]
soma=0
for i in range(3):
    print("Valor do elemento da posição " + str(i), " = " , notas[i])
    soma = notas[i] + soma

media = soma/len(notas)
print("Media da turma = ", media)
```

Listagem 2

Exemplo Vetor 02

Definir um vetor “nota” de tamanho 5 de tipo inteiro. nota = [60, 95,80, 50, 98]

Índice	Valor		0	1	2	3	4
0	60	nota[0]	60	95	80	50	98
1	95	nota[1]					
2	80	nota[2]					
3	50	nota[3]					
4	98	nota[4]					

- ✓ Os valores 60, 95, 80, 50, 98 correspondem as notas, isto e, aos elementos do vetor.
- ✓ Os valores 0, 1, 2, 3, 4 correspondem aos índices.

```
nota = [60, 95,80, 50, 98]

for i in range(len(nota)):
    print("Valor do elemento da posição " + str(i), " = " , nota[i])

print()
print("Elemento que está na posição 3 = ", nota[3])
```

Listagem 3

Manipulando elementos e índices

- ✓ A manipulação do vetor depende da manipulação dos índices.
- ✓ Por exemplo, imprimir a quarta nota do vetor nota:
- ✓ `print (nota [3]) = 50`
- ✓ nota é a variável.
- ✓ 3 é o índice.
- ✓ `[]` é o operador de elemento.
- ✓ Lê-se: Acessando a variável nota na posição de índice 3.

Erros Comuns

- ✓ Acessar um índice inválido.
- ✓ `print ( nota [ 5 ] ) # Não existe o índice 5`
Traceback (most recent call last):
File "c:\Users\SERGIOMEDINA\GoogleDrive\Lab-IPE-Python2023\Aula07-E_Dados-Python\vetor1.py", line 11, in <module>
print("Elemento que está na posição 5 = ", nota[5])
IndexError: list index out of range
- ✓ Não colocar o índice.
- ✓ `print ( nota ) # Qual Posição???`
- ✓ Não foi especificado o índice;
- ✓ Em Python, imprime a lista inteira.

b) Variáveis do tipo lista (mutáveis)

Lista é uma sequência finita de elementos ou também conhecida como uma coleção de objetos de um **mesmo tipo (homogêneo) ou tipos diferentes (heterogêneo)** .

Estrutura de dados constituída por uma sequência ordenada e finita de itens (os quais podem, inclusive, ser outras listas, ditas sublistas), e que pode ser modificada com a inserção, exclusão e reordenamento dos itens.

São **mutáveis** e podem ser alteradas a qualquer momento. A lista uma estrutura de dados amplamente utilizada no desenvolvimento de software.

As listas são conjuntos de itens separados por **vírgulas (,)** e delimitados por **colchetes []**.

Similarmente às strings, acessamos os itens de uma lista através dos operadores `[]` e `[:]` , e o operador `+` concatena duas listas.

Estrutura de dados mutável onde a ordenação natural é estabelecida pela ordem de entrada.

O último item de uma lista não ordenada SEMPRE será o último elemento adicionado

As listas são classificadas conforme o ordenamento dado aos seus elementos. Então, podemos dizer que existem infinitos tipos de listas, até porque, existem infinitas maneiras de ordenarmos e organizarmos os itens de uma lista, sejam eles iguais, sejam diferentes.

O conceito de lista é intuitivo e a forma mais primitiva de organização de conjuntos de elementos. Todo dia temos uma lista de afazeres e no final do dia, uma lista de tarefas realizadas. Temos listas de telefone, de amigos, inimigos, de produtos a serem vendidos, listas do que deve ser comprado, do que pode e não pode ser feito e etc.

- lista de e-mails
- lista de telefones
- lista de telefones importantes
- listas dos sites que acompanhamos
- lista de e-mails recebidos
- listas de músicas a serem executadas
- lista dos nossos filmes prediletos
- lista de impostos e tributos
- lista de pagamentos futuros

Exemplos de listas

Todo programa é uma lista de instruções que serão executados numa ordem determinado pelo programador. Por isso, a lista é um dos conceitos mais utilizados na computação até porque, a principal tarefa do computador é processar listas.

PRINCÍPIOS DAS LISTAS

Toda lista pode conter **0, n** ou **infinitos elementos**. A lista é uma Estrutura de Dados propriamente dita, e os elementos que a compõem, podem ser qualquer um, inclusive outras listas.

A lista é uma

estrutura de dados homogêneos (armazena dados do mesmo tipo)

- o notas = [10.0, 5.4, 6.7, 2.2, 7.5]

e ao mesmo tempo heterogêneos (armazena dados de diferentes tipos)

- o lista = ['A', 1, 2, 'Casa', 2.3]

Declaração e Criação de LISTAS

A declaração é feita com o uso de colchetes, separando os itens por vírgulas.

- ✓ Lista vazia, ou seja, sem elementos:
- ✓ `Lista_vazia = [ ]`
- ✓ Lista com elementos inteiros:
 - o `Lista_inteiros = [2, 4, 6, 8, 10]`
- ✓ Lista com elementos reais (float):
 - o `Lista_reais = [9.0, 10.0, 8.5, 7.8]`
- ✓ Lista com elementos string:
 - o `Lista_frutas = ["abacaxi", "pera", "uva", "abacate"]`

- ✓ Lista de inteiros com tamanho 5 preenchida por zeros:
 - `Lista_zerada = [0]*5`
- ✓ Lista de strings com tamanho 4 preenchida com strings vazias:
 - `Lista_strings = [""]*4`

Indexação

É possível acessar os valores individualmente através da indexação.

```
lista[1]
```

O índice pode também ser negativo, acessando a lista a partir do fim.

```
lista[-1]
```

Fatiamento

O fatiamento retorna uma sublista a partir do intervalo definido.

```
lista[inicio:fim]
lista[inicio:fim:incremento]
lista[inicio:]
lista[:fim]
lista[:,incremento]
```

Mutabilidade

A lista é uma sequência *mutável*, o que significa que podemos alterar itens seus individualmente:

```
lista[1] = 0
```

Além da alterar um item, podemos também removê-lo.

```
del lista[1]
```

Concatenação

Permite a união de duas listas.

```
[1, 2] + [3, 4]
```

Aninhamento

É possível também gerar uma lista de listas, o que é o equivalente a uma matriz em Python.

```
matriz = [
    [1, 2],
    [3, 4]
]
```

Cópia

Um detalhe importante sobre o funcionamento de listas é que, ao copiarmos uma lista, na realidade estamos fazendo uma referência para a lista original. Assim, se alterarmos a lista original, alteramos também a cópia.

MANIPULAÇÃO DE LISTAS (Normas gerais das listas)

- 1) Novos itens sempre serão adicionados após o último item,
- 2) O primeiro item adicionado a lista sempre será o primeiro elemento da estrutura.

Uma lista no python funciona como uma lista da vida real. Onde podemos incluir, alterar, excluir elementos da lista

A lista contém 4 conceitos fundamentais (**CRUD**):

- a) **Adição**
- b) **Alteração [da disposição dos elementos]**
- c) **Exclusão**
- d) **Leitura**

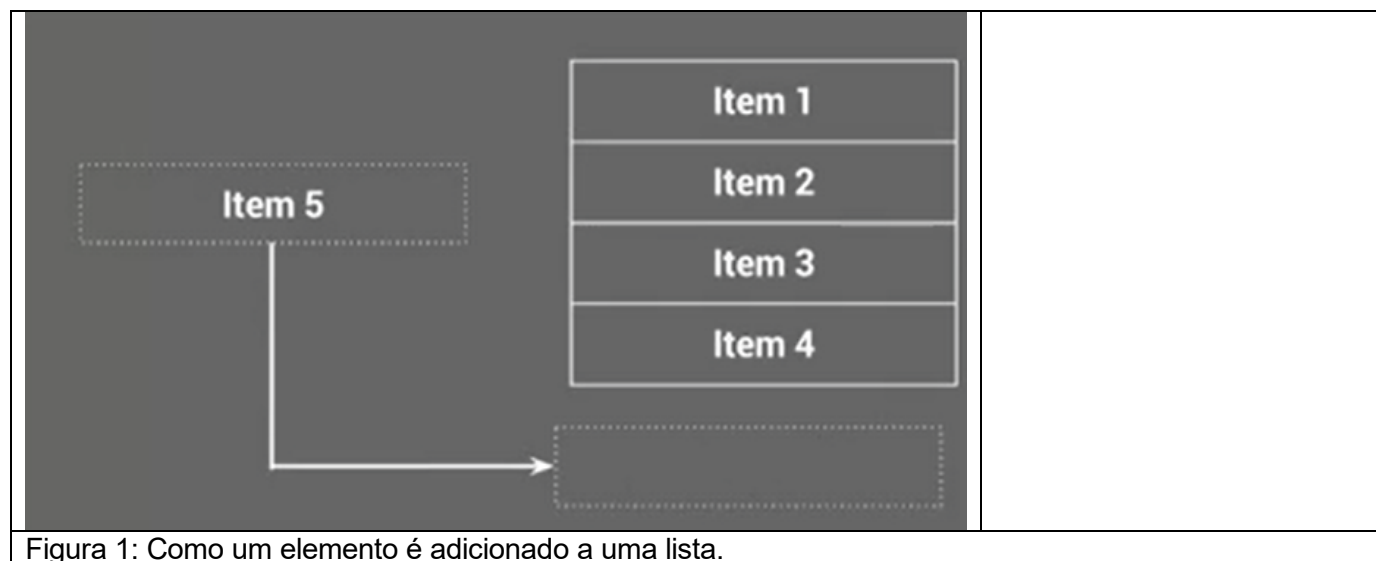


Figura 1: Como um elemento é adicionado a uma lista.

Nem toda lista permitirá a adição, alteração ou exclusão de itens, porém, a capacidade de leitura é obrigatória.

É importante entender que a lista possui um conjunto de propriedades e funções que a definem e, **todo** elemento contido nesta lista também poderá ter um conjunto de funções.

TIPOS DE LISTA

O conceito mais simples de lista é aquele onde temos uma série de elementos. Os vários tipos de definições de lista dizem respeito a forma como os elementos são organizados, ou então, as ações conjuntas ligadas a inclusão e remoção de elementos.

Existem duas maneiras de declarar uma lista no python. A primeira basta nós atribuirmos um conjunto de elementos a uma variável:

Uma lista sempre vai estar delimitada entre dois **colchetes []**

```
Lista2 = [1,2,3,4,10,8,9, 5, 0]
print("Elementos da lista 2")
print(lista2)
type(lista2)
```

Listagem 1

Vamos colocar um conjunto de elementos dentro de parênteses retos como parâmetro da função **type()** para vermos o tipo de objeto:

```
type(lista2)
```

Elementos de uma lista

Para retornarmos um elemento de uma lista basta informarmos qual é o índice do elemento junto com a variável.

Estrutura: `variavel[indice]`

Lista2[0] = 0	✓ retorna o primeiro elemento da lista que tem o índice
Lista2[4] = 10	✓ retorna o terceiro elemento da lista que tem o índice cinco, lembrando sempre que o primeiro elemento de uma lista sempre tem como índice o número inteiro zero.

Veja a tabela A de relação índice, Elemento e valor que da nossa lista **lista2 = [1, 2, 3, 4, 5, 10, 0, 6, 7]**

Índice	0	1	2	3	4	5	6
Elemento	1º	2º	3º	4º	5º	6º	7º
Valor	1	2	3	4	5	6	7

Tabela A

Podemos fazer uma operação matemática acedendo os elementos da nossa lista.

```
print("Soma em uma lista ", (lista2[0] + lista2[4]))
```

Temos que adicionamos o valor do **índice zero** ao valor do **índice cinco**. O **índice zero** da nossa **variável lista** contém o **valor 1** e o **índice 5** o **valor 10**, então **1+10=11**.

Podemos criar uma lista com caracteres ou conjunto de caracteres.

```
lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]
```

Vamos agora retornar os elementos da nossa lista:

```
print("Elemento da 1ra posição =", lista_caracteres[0])
print("Elemento da 2da posição =", lista_caracteres[1])
print("Elemento da 3ra posição =", lista_caracteres[3])
print("Elemento da 6ta posição =", lista_caracteres[5])
```

Outra forma de declarar lista no python é atribuindo a uma variável o objeto **list**

```
lista2= list("Python")
print ("Elemento da lista 2 =",lista2)
```

No exemplo acima definimos uma variável e atribuímos um objeto do tipo **list** e como parâmetro passamos um objeto iterável. Como parâmetro passamos uma string, uma string é uma lista de caracteres, ou seja, um objeto iterável.

Imagina que queiramos fazer com que a cadeia de caracteres “Python” seja o único elemento de uma lista e não a lista.

No exemplo acima “Python” é a lista e os caracteres os elementos, mas o que queremos é tornar o objeto “Python” um elemento único de uma lista.

Para isso, iremos usar uma vírgula depois de “Python”.

```
lista3= list(("Python",))
print ("Elemento da lista 3 =", lista3)
```

Como pode ver, “Python” agora é o último elemento da nossa lista.

Listas de listas, Listas contidas em listas ou sub-listas

Uma lista é um objeto e cada elemento contido numa lista é um objeto, sendo a lista um objeto, ela pode fazer parte de uma outra lista.

Se executarmos o código acima, diremos ao Python que queremos acessar o elemento 0 do objeto de nome "lista" e em seguida, queremos acessar um elemento de índice 0, do elemento que nos será retornado.

```
lista4 = [[1,2,3,4,10,8,9, 5, 0], list("Python"), ["a", "b", "c", "d", "e", "f", "g"], list((100, 200, 300))]
```

Listagem2:

```
lista4 = [[1,2,3,4,10,8,9, 5, 0],
list("Python"),
["a", "b", "c", "d", "e", "f", "g"],
list((100, 200, 300))]
```

Listagem3:

Como pode ver, temos uma lista delimitada por colchetes [] e dentro dessa lista colocamos vários parênteses retos, ou seja, adicionamos como elementos de uma lista, várias listas.

Acedendo elementos de uma lista de listas

Elementos da lista 4

```
[[1, 2, 3, 4, 10, 8, 9, 5, 0], ['P', 'y', 't', 'h', 'o', 'n'], ['a', 'b', 'c', 'd', 'e', 'f', 'g'], [100, 200, 300]]
```

Elemento da 1ra lista = [1, 2, 3, 4, 10, 8, 9, 5, 0]

Elemento da 2da lista = ['P', 'y', 't', 'h', 'o', 'n']

Elemento da 3ra lista = ['a', 'b', 'c', 'd', 'e', 'f', 'g']

Elemento da 4to lista = [100, 200, 300]

Listagem 4:

No índice zero da lista temos o elemento **[1,2,3,4,10,8,9, 5, 0]** que é uma lista, no índice 1 temos o elemento **[['P', 'y', 't', 'h', 'o', 'n']]** que é uma lista definida pelo objeto list onde levou como parâmetro a cadeia de caracteres iteráveis "Python", no índice 3 temos o elemento **[100, 200, 300]** que também é uma lista.

O acesso a elementos a listas contidas em outras, faz-se através da concatenação de pares de colchetes. Logo, quando mais aninhado tiver a lista, maior a concatenação de colchetes necessária. É importante destacar que cada par de colchete representa uma lista.

lista[0][0]

Os elementos dessas **sub-listas** também podem ser acedidos indicando o índice da **sub-lista** na lista principal e o índice do elemento da **sub-lista**.

Vamos agora aceder o segundo elemento da nossa sub-lista de **índice 0**, ou seja, do primeiro elemento da nossa lista.

```
print("Elementos da 1ra lista =", lista4[0])
print("Acessando ao 6to elemento da 1ra lista =", lista4[0][5])
```

```
Elementos da 1ra lista = [1, 2, 3, 4, 10, 8, 9, 5, 0]
Acessando ao 6to elemento da 1ra lista = 8
```

Listagem 5:

Métodos

Listas suportam também operações especiais que chamamos de *métodos*. Para usar um método, usamos o nome da nossa lista seguida do método e seus parâmetros.

Abaixo temos uma lista de alguns dos métodos de listas:

Método	Resultado
.index(item)	Retorna o índice de um item.
.append(item)	Insere um item do final da lista.
.insert(pos, item)	Insere um item na posição indicada.
.append(seq)	Insere os item da sequência do fim da lista.
.remove(item)	Remove a primeira ocorrência do item.
.pop(pos)	Retorna e remove um item da lista.
.copy()	Retorna uma cópia da lista
.sort()	Ordena a lista.
.reverse()	Inverte a ordem dos itens.
.count(item)	Retorna o número de ocorrências de um item.

Iteração

Como toda sequência, listas podem ser iteradas com o uso de for.

```
for item in lista
```

No exemplo, a variável `item` assumirá o valor de um novo item da lista a cada iteração. Podemos também usar a função **`enumerate()`** para obter simultaneamente os índices e valores dos itens quando necessário.

```
for item, valor in enumerate(lista)
```

Outras funções incluem, por exemplo, a iteração em ordem reversa.

```
for item in reversed(lista)
```

Pertencimento (`in` e `not in`)

Python possui um operador exclusivo para verificar a existência de um item em listas (e outras estruturas) sem necessidade do uso de iteração.

```
2 in [0, 1, 2, 3]
```

É possível também usar o operador `in` negado.

```
4 not in [0, 1, 2, 3]
```

Funções para trabalhar com qualquer tipo de lista.

1-length ou len

A função `length` retorna para nós o tamanho da nossa lista, no python não precisamos digitar a palavra ***length***, basta apenas o ***len***.

EX:

```
lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]

print("Elementos da lista de caracteres =", lista_caracteres[0])
print("Tamanho da lista de caracteres =", len(lista_caracteres))
```

Listagem 6:

Incluir, alterar e excluir elementos de uma lista

Incluir elemento Usando a função **`append()`**

Para adicionar diretamente um elemento a uma lista usamos a função **`append`**

```
lista_caracteres.append("t")
```

```
lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]

print("Elementos da lista de caracteres =", lista_caracteres)
print("Tamanho da lista de caracteres =", len(lista_caracteres))
print("Incluir um elemento no final da lista de caracteres ")
lista_caracteres.append("t")
```

Listagem 7:

Veja a diferença, **o append adiciona elementos a uma lista e o sinal de adição faz concatenação de uma lista.**

```
lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]
lista_numeros = [100, 400, 300, 700, 800]

print("Elementos da lista de caracteres =", lista_caracteres)
print("Tamanho da lista de caracteres =", len(lista_caracteres))
print("Incluir um elemento no final da lista de caracteres ")
lista_caracteres.append("t")
print("Nova lista de caracteres =", lista_caracteres)
print()
print("Elementos da lista de caracteres =", lista_numeros)
print("Novo tamanho da lista de caracteres =", len(lista_numeros))
print("Incluir um elemento no final da lista de numeros")
lista_numeros.append(7000)

print("Nova lista de caracteres =", lista_numeros)
print("Novo tamanho da lista de caracteres =", len(lista_numeros))
print()
```

Listagem 8:

Incluir elemento em uma determinada posição Usando a função *insert(index, element)*

• **insert (i, e):** Para adicionar elemento em uma determinada posição invocamos a *insert()* passando como parâmetro a posição em que queremos inserir um elemento e o elemento.

```
lista_numeros.insert(2, 8000) # insere o elemento 8000 na posição 2 de lista_numeros.
```

Limpendo uma lista usando a função *clear()*

```
lista5.clear()
```

Removendo elementos de uma lista, *remove(element)*

Para além de adicionar elementos de uma lista, podemos também remover elementos de uma lista usando a função *remove()* desde que a lista seja mutável.

listaX. remove (e): remove a primeira ocorrência do elemento e em ListaX; ocorre exceção *ValueError* se e não ocorrem na listaX (exceções não serão abordadas neste curso). Por exemplo:

```
lista_numeros.remove(400)
```

Deletando um determinado elemento da lista a função *pop()*

Por default a função *pop()* elimina o último elemento da lista, para eliminar um determinado elemento basta na função *pop* passarmos como parâmetro o índice do elemento que queremos eliminar

```
lista_numeros.pop()
```

Usando a função *count(element)*

• **listaX. count (e):** retorna o número de ocorrências de na listaX. Por exemplo:

```
print(" Retorna o numero de ocorrencia do elemento 100 =" ,lista_numeros.count(100))
```

Usando a função `index(element)`

• **listaX. index (e):** retorna o índice do elemento e da listaX; ocorre exceção `ValueError` se e não ocorre em x (exceções não serão abordadas neste curso). Por exemplo:

```
print("Retorna a posição do elemento 700 = ", lista_numeros.index(700))
```

Usando a função `reverse()`

• **x. reverse ():** inverte a ordem dos elementos em x. Por exemplo:

```
lista_numeros.reverse()  
print("Retorna a lista invertida = ", lista_numeros)
```

Ordenamento de listas

Vamos ordenar a lista de forma ascendente

Para ordenar uma lista de forma ascendente usamos o método `sort()` (`sort=ordenar`)

Usando a função `sorted(listX)`

• **sorted(x):** retorna x em ordem crescente, ordena em ordem crescente os elementos em x (equivalente a `x = sorted(x)`); é preciso que exista uma operação de comparação definida entre os elementos da lista, caso contrário ocorre um erro de tipo durante a execução. Por exemplo:

```
sorted(lista_numeros)  
print("Lista em ordem crescente", sorted(lista_numeros))
```

E se por acaso quisermos ordenar decendentemente?

Podemos ordenar a nossa lista de forma decendente usando primeiro o método `sort()` e depois a função `reverse()`. Se a lista já estiver ordenada ascendentemente basta apenas o método **reverse**.

Vamos reiniciar a lista e mostrar como se faz.

```
lista_numeros = ['B', 'A', 'I', 'C', 'E', 'H', 'D', 'F', 'G']  
lista_numeros.sort(reverse=True)
```

Usando a função `max()` e `min()`

• **min(x), max(x):** retorna o menor (maior) elemento de x; é preciso que exista uma operação de comparação definida entre os elementos da lista, caso contrário ocorre um erro de tipo durante a execução.

Por exemplo:

```
min(lista_caracteres)  
max(lista_caracteres)  
print("Numero minimo da Lista = ", min(lista_caracteres))  
print("Numero maximo da Lista = ", max(lista_caracteres))
```

Usando a função `sum()`

• **sum(x):** soma dos valores em x: soma dos valores em x; é preciso que exista uma operação de soma (+) entre os valores de x, caso contrário ocorre um erro de tipo durante a execução. Por exemplo:

```
sum(lista_numeros))  
print("Soma de todos os elementos da Lista = ",sum(lista_numeros))
```

O TIPO DOS ELEMENTOS

Estudamos listas contém só e somente só objetos, dessa forma, cada objeto pode ter sido criado através de uma classe diferente e assim, conter informações das mais diversas.

No código abaixo, estamos declarando uma variável e adicionando a esta uma lista que contém elementos de vários tipos.

```
lista_mista = ["d", True, 1,2,3,4, "Testando o Python"]
```

No código acima, definimos inicialmente, uma variável de nome lista_mista e atribuímos a mesma uma lista contendo 7 elementos de tipos diferentes. O primeiro elemento é um caractere, o segundo é um valor lógico, o terceiro é um número e assim por diante.

c) Estrutura de dados tipo Tuplas (são imutáveis)

As tuplas são semelhantes às listas, porém **são imutáveis**. Não se pode modificar nenhum elemento de uma tupla.

A Tupla pode conter 0 ou “n” elementos e estes não poderão ser alterados.

Não se pode acrescentar, **deletar** ou fazer atribuição aos seus itens. Embora a tupla possa conter elementos mutáveis, esses elementos não podem sofrer atribuições porque isso modificaria a referência ao objeto.

Tuplas uma estrutura de **dados heterogêneos**, ou seja armazena dados de **tipos distintos**.

No entanto, podemos também obter acesso aos elementos de uma tupla com indexação, sendo que os índices iniciam com 0. Por exemplo: tupla [0] retorna o primeiro elemento de tupla.

Os elementos de uma tupla são também separados por vírgulas, porém são delimitadas por parênteses, diferentes das listas, que são delimitadas por colchetes.

Uma tupla (dupla, tripla, quádrupla etc.) é uma sequência de valores distinguidos de acordo com a posição em que aparecem na tupla.

A diferença entre lista e tupla é que a lista é uma **estrutura de dados homogêneos** e Tuplas uma estrutura de dados heterogêneos.
Geralmente numa lista, o tipo de dados da sequência é do mesmo tipo e numa tupla, geralmente são de **tipos distintos**.

As tuplas podem receber qualquer tipo de dado. A tupla é uma **estrutura de dados heterogênea**, em que seus elementos não são necessariamente do mesmo tipo.

Em uma tupla, os valores componentes são separados por vírgula. Por exemplo:

```
dupla = ( " abc " , 3)  
tripla = (1 , True , " 1 " )
```

Como saber se um determinado valor está contido na nossa lista? Podemos saber com os operadores IN e NOT IN

Operador IN

O operador IN serve para verificar se um valor está contida numa lista.

X in [...] — Verifica se o valor da variável x está contida na lista e não se a instância da variável x está contida na lista.

O teste de pertinência (in em Python) pode ser usado com tuplas em Python (e com qualquer coleção: listas, cadeias de caracteres, dicionários). Por exemplo:

```
3 in dupla  
retorna True.
```

```
tupla2 = range(1,6)  
if 6 in tupla2:  
    print("Valor contido da tupla")  
else:  
    print("Valor Não contido da tupla")
```

Listagem

Operador NOT IN

O **not in** faz o contrario do **in**. O que o **in** retorna True o **not in** retorna False.

```
8 not in dupla  
retorna false.
```

```
tupla2 = range(1,6)  
if 6 not in tupla2:  
    print("Valor não contido da tupla")  
else:  
    print("Valor contido da tupla")
```

Listagem

Tuplas

Tuplas são sequências similares a listas, com a principal diferença que seus itens não podem ser alterados individualmente. Por isso, dizemos que são sequências *imutáveis*.

Declaração

A declaração de tuplas é feita com o uso de parênteses.

```
tupla = ()  
tupla = (1,)  
tupla = (1, 2, 3)  
tupla = 1, 2, 3
```

Operações

Tuplas suportam a maioria das operações de listas, como **indexação**, **fatiamento**, **concatenação**, **aninhamento**, **iteração** e **pertencimento**.

Métodos

Ao contrário das listas, as tuplas **não suportam métodos**.

As tuplas são úteis quando precisamos que uma **sequência não seja alterada**. Por não possuírem as mesmas limitações em relação a cópia, são úteis também na passagem de sequências como parâmetros de funções, como veremos mais adiante.

Strings

Vimos anteriormente que strings são um tipo de dado que representa texto, mas strings são também tratadas como sequências de caracteres. Em Python, não existe diferença de um caractere para uma string.

Assim como as tuplas, strings são sequências **imutáveis**.

Operações

Strings suportam a maioria das operações das demais sequências, como indexação, fatiamento, concatenação e iteração.

Métodos

Abaixo temos uma lista de alguns dos métodos de strings:

Método	Resultado
.upper()	Retorna cópia da str em maiúsculas
.lower()	Retorna cópia da str em minúsculas
.capitalize()	Retorna cópia da str com a primeira letra maiúscula
.title()	Retorna cópia com cada palavra iniciando em maiúscula
.isalpha()	Retorna True se a str é composta de letras
.isdigit()	Retorna True se a str é composta de números
.isalnum()	Retorna True se a str é composta de letras e números
.islower()	Retorna True se a str é minúscula
.isupper()	Retorna True se a str é maiúscula
.find(sub)	Retorna o número de ocorrências da substring
.count(sub)	Retorna o índice da substring
.replace(old, new, n)	Substitui os termos n vezes
.strip(char)	Remove caracteres no início/fim da str (' ' por padrão)
.startswith(sub)	Compara o início da string, retorna True/False
.endswith(sub)	Compara o final da string
.split()	Retorna lista com palavras convertidas em itens
'd'.join(lista)	Retorna str com itens separados pelo delimitador

Strings multilinha

São delimitadas por três aspas, e permitem que strings possuam diversas linhas. Regras de indentação são ignoradas no interior das strings multilinha.

```
print("""Olá,  
  
este é um exemplo de strings multilinha.  
  
- Grupy-Lavras  
  
""")
```

Formatação de strings

É um recurso que permite criar formatos customizados para strings, além de strings compostas usando variáveis, que são úteis quando seu conteúdo é dinâmico.

Para isso usamos marcadores {} e o método .format().

Daremos exemplos de alguns dos usos comuns, mas um estudo detalhado do assunto está fora do escopo deste curso (a sintaxe de formatação é extensa e considerada uma *mini-linguagem*)

```
# uso de marcadores  
'{} passo depois do {}'.format('um', 'outro')  
# uso de marcadores com índices  
'{1} passo depois do {0}'.format('um', 'outro')  
# preenchimento de números com zeros  
'{:03d}'.format(1)  
# formatação de casas decimais  
'{:2f}'.format(3.141592653589)
```

Quando usamos valores numéricos como parâmetros, eles são automaticamente convertidos em strings. É importante mencionar também que Python suporta não uma, mas duas sintaxes de formatação, conhecidas como *nova* e *antiga*. A sintaxe nova é preferível e foi introduzida na versão 3, que suporta também a formatação antiga para fins de compatibilidade.

Glossário

atribuição (*assignment*)

Comando que atribui um valor a uma variável.

avaliar (*evaluate*)

Simplificar uma expressão através da realização de operações, para produzir um valor único.

comando (*statement*)

Trecho de código que representa uma instrução ou ação. Até agora, os comandos vistos foram de atribuição e exibição.

comentário (*comment*)

Informação em um programa dirigida a outros programadores (ou qualquer pessoa que esteja lendo o código fonte) e que não tem efeito na execução do programa.

composição (*composition*)

Habilidade de combinar expressões e comandos simples em expressões e comandos compostos, de forma a representar operações complexas de forma concisa.

concatenar (*concatenate*)

Juntar dois operandos lado a lado.

diagrama de estado (*state diagram*)

Representação gráfica de um conjunto de variáveis e os valores aos quais elas se referem.

divisão inteira (*integer division*)

Operação que divide um inteiro por outro e resulta em um inteiro. A divisão inteira resulta no número de vezes que o numerador é divisível pelo denominador e descarta qualquer resto.

expressão (*expression*)

Combinação de variáveis, operadores e valores, que representa um resultado único.

operando (*operand*)

Um dos valores sobre o qual o operador opera.

operador (*operator*)

Símbolo especial que representa uma computação simples, como adição, multiplicação ou concatenação de strings.

palavra-chave (*keyword*)

Palavra reservada usada pelo compilador/interpretador para analisar o programa; você não pode usar palavras-chave como `if`, `def`, e `while` como nomes de variáveis.

ponto-flutuante (*floating-point*)

Formato para representar números que possuem partes fracionárias.

regras de precedência (*rules of precedence*)

O conjunto de regras que governa a ordem em que expressões envolvendo múltiplos operadores e operandos são avaliadas.

tipo (*type*)

Um conjunto de valores. O tipo de um valor determina como ele pode ser usado em expressões. Até agora, os tipos vistos são: inteiros (tipo `int`), números em ponto-flutuante (tipo `float`) e strings (tipo `string`).

valor (*value*)

Um número ou string (ou outra coisa que ainda vamos conhecer) que pode ser atribuída a uma variável ou computada em uma expressão.

variável (*variable*)

Nome que se refere a um valor.

Bibliografia

Python, Escreva seus primeiros programas, Felipe Cruz, Casa do Código.

Curso Introdutório de Python, Grupy-Sanca, 09 de maio de 2019

Tutorial Python, Release 2.4.2. Guido van Rossum, Fred L. Drake, Jr., editor. Tradução: Python Brasil

Curso Python 3. Juracy Filho e Leonardo Leitão. Cod3r.com.br

https://aprendacompy.readthedocs.io/pt/latest/capitulo_01.html

<https://www.devmedia.com.br/python-trabalhando-com-variaveis/38644>

<https://cadernodelaboratorio.com.br/2017/12/01/variaveis-em-python3/>

Parte 2 – Prática (mão na massa)!!!

Para as atividades práticas vamos utilizar o VS Code, que deve estar previamente configurado para executar o python)

Para a criação dos arquivos (com extensão .py) vamos utilizar o editor de texto

Para executar, deve utilizar o interpretador do VS Code, temos 2 opções

- Apertar o Botão direito e selecionar a opção **RUN PYTHON TERMINAL**
- No menu superior, selecionar Debug -> **START WITHOUT DEBUGGING**

Para limpar o console, utilizar o comando **clear** ou **cls**

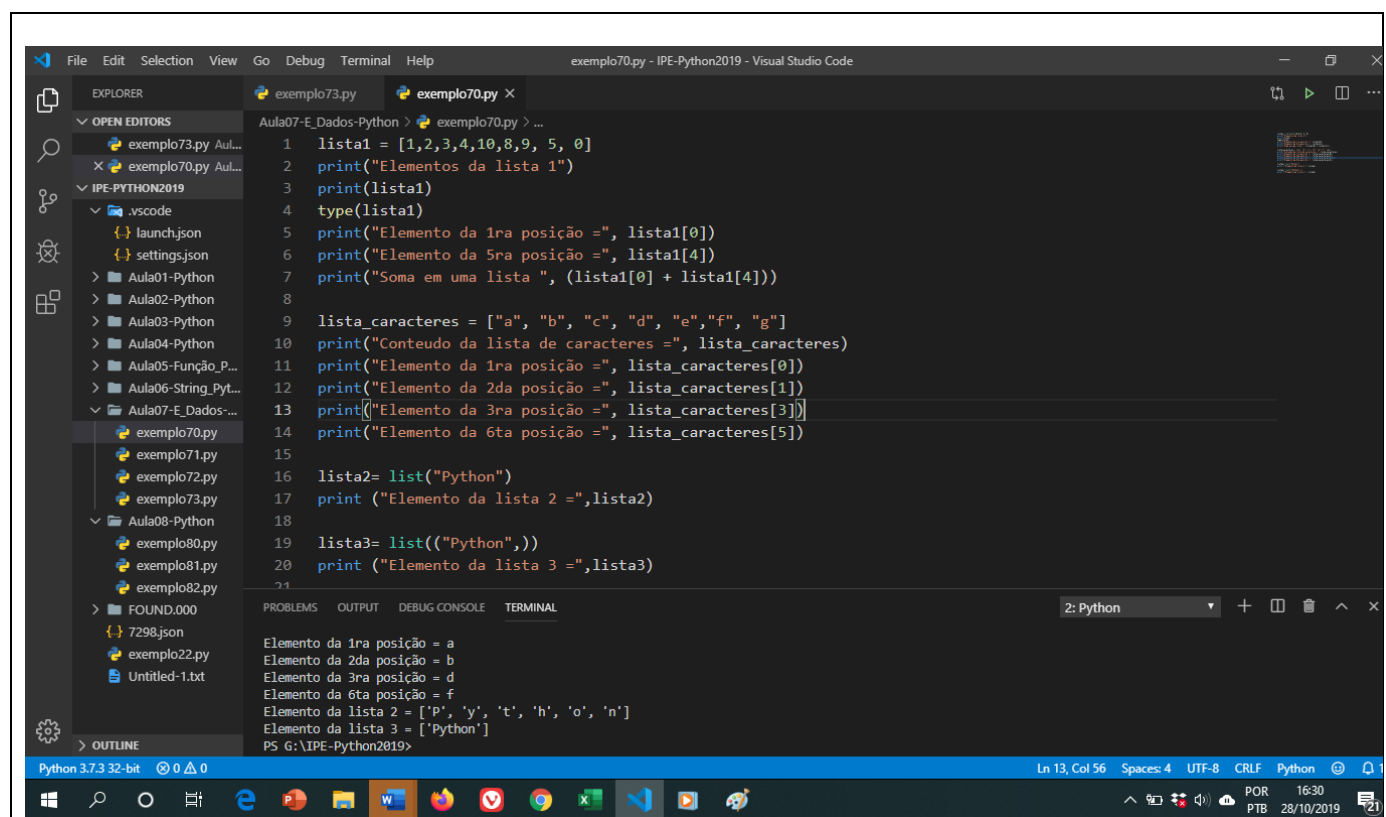
2) Criação do Projeto no VS CODE

- 1) Para isto crie uma pasta no seu pendrive com o nome **IPE-Python-2023**.
- 2) Dentro do **IPE-Python-2023**, criar a pasta **Aula07-Python**
- 3) Utilizando o VS CODE acesse a pasta criada (**IPE-Python-2023/ Aula07-Python**)
- 4) Para isso, acione a opção **File > New > File** e crie os seguintes (exemplo70.py, exemplo71.py, exemplo72.py, exemplo73.py e exemplo74.py), para melhor organização (**BOA PRATICA!!!**)

Exemplo 70

PASTA: IPE-Python-2023/ Aula07-Python

arquivo: exemplo70.py



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays a project named 'IPE-PYTHON2019' with a folder 'Aula07-E-Dados-...' containing several Python files, including 'exemplo70.py'. The main editor window shows the code for 'exemplo70.py' with the following content:

```
1 lista1 = [1,2,3,4,10,8,9, 5, 0]
2 print("Elementos da lista 1")
3 print(lista1)
4 type(lista1)
5 print("Elemento da 1ra posição =", lista1[0])
6 print("Elemento da 5ra posição =", lista1[4])
7 print("Soma em uma lista ", (lista1[0] + lista1[4]))
8
9 lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]
10 print("Conteúdo da lista de caracteres =", lista_caracteres)
11 print("Elemento da 1ra posição =", lista_caracteres[0])
12 print("Elemento da 2da posição =", lista_caracteres[1])
13 print("Elemento da 3ra posição =", lista_caracteres[3])
14 print("Elemento da 6ta posição =", lista_caracteres[5])
15
16 lista2= list("Python")
17 print ("Elemento da lista 2 =",lista2)
18
19 lista3= list(("Python",))
20 print ("Elemento da lista 3 =",lista3)
21
```

The bottom of the image shows the Windows taskbar with various application icons and the system clock indicating 16:30 on 28/10/2019.

Figura A: exemplo70.py

Exemplo 71

PASTA: IPE-Python-2023/ Aula07-Python

arquivo: exemplo71.py

```
lista01 = [10, 20, 30, 40, 50]
lista02 = [60, 70, 80, 90, 100, 110, 120]

print("Elementos da lista 01 = ", lista01)
print("Tamanho da lista 01 = ", len(lista01), "; e Tipo : ",type(lista01))
print()
print("Elementos da lista 02 = ", lista02)
print("Tamanho da lista 02 = ", len(lista02), "; e Tipo : ",type(lista02))
print()
print("Elemento da 1ra posição =", lista01[0])
print("Elemento da 5ta posição =", lista01[4])
print("Soma em uma lista ", (lista01[0] + lista01[4]))

lista03 = lista01 + lista02 # concatenando 2 listas
print("Soma dos elementos de 2 listas (lista01 + lista02) = ", lista03 )
print("Tamanho da lista 03 = ", len(lista03)," ; e Tipo : ",type(lista03))

lista04 = lista01 *2
print("Multiplicando os elementos da lista01 = ", lista04 )

lista05= list("Python")
print ("Elemento da lista 5 =",lista05)

lista06= list(("Python 3.7",))
print ("Elemento da lista 3 =",lista06)

lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]

print("Conteudo da lista de caracteres =", lista_caracteres)
print("Elemento da 1ra posição =", lista_caracteres[0])
print("Elemento da 2da posição =", lista_caracteres[1])
print("Elemento da 3ra posição =", lista_caracteres[3])
print("Elemento da 6ta posição =", lista_caracteres[5])

del lista04 # Exclui a lista04
print("Lista 04 = ", lista04)
```

Figura B: exemplo71.py

Exemplo 72

PASTA: IPE-Python-2023/ Aula07-Python
arquivo: exemplo72.py

```
# Iteração
"""
lista_num = [100, 200, 300, 400, 500]

for item in lista_num: # iteração com for
    print("Elementos da lista, lista_num", item)
"""

num = 5
lista_num = [500, 200, 300, 400, 100]
lista_nomes= ["Wladimir", "Carlos", "Andre", "Ana", "Anastacia"]

print("Tamanho da Lista inicial, lista_num = ", len(lista_num))
print("Elementos da lista original lista_num = ", lista_num)

print ("Lista em ordem crescente da lista_num= ", sorted(lista_num))
print ("Lista em ordem alfabetica = ", sorted(lista_nomes))

for indice, elemento in enumerate(lista_num): #obter simultaneamente os índices e valores dos itens
    quando necessário.
    lista_num[indice] +=1000 # lista_num[indice] = lista_num[indice] +1000
    print("Posição ", indice+1, "Elemento da lista : ", lista_num[indice] )

print("Tamanho da Lista final, lista_num = ",lista_num)
print("Tamanho da Lista final, lista_num = ", len(lista_num))

print("1ro elemento da = ",lista_num[0])
print("2ro elemento da = ",lista_num[1])
print("3ro elemento da = ",lista_num[2])

lista_num = [100, 200, 300, 400, 500]

for elemento in reversed(lista_num):
    print(elemento)

print("Lista reversa = ", lista_num)
```

Figura C: exemplo72.py

Exemplo 73

PASTA: IPE-Python-2023/ Aula07-Python
arquivo: exemplo73.py

```
#Listas de listas ou sub-listas
lista1 = [ [],[],[],[],[] ] # Definição de uma lista contendo 3 sublistas como elementos
lista2= [
    ["a","b","c","d"],
    [5.4,6.5,7.3],
    [3, 4, 6, 7, 5],
    []
]
#lista2= [ ["a","b","c","d"], [5.4,6.5,7.3], [3, 4, 6, 7, 5], [ ] ]

lista_mista = ["d", True, 1,2,3,4, "Testando o Python"]

print("1ra sublista contida na lista02 =", lista2[0])
print("3ra sublista contida na lista02 =",lista2[2])
print("4ra sublista contida na lista02 =",lista2[3])
print()
print("2do elemento da sublista contida na lista02 =", lista2[0][1])
print("3ro elemento da sublista contida na lista02 =", lista2[1][2])
print()

print("Tamanho da lista 1 = ",len(lista1))
print("Tamanho da lista 2 = ",len(lista2))
```

Figura D: exemplo73.py

Exemplo 74

PASTA: IPE-Python-2023/ Aula07-Python

arquivo: exemplo74.py

```
lista4 = [
    [1,2,3,4,10,8,9, 5, 0],
    list("Python"),
    ["a", "b", "c", "d", "e", "f", "g"],
    list((100, 200, 300))
]
# ["Python"] = list("Python")
print("Elementos da lista 4", lista4)
print(" Tamanho da lista 4 = ", type(lista4))

print("Elementos da 1ra lista =", lista4[0])
print("Elementos da 2da lista =", lista4[1])
print("Elementos da 3ra lista =", lista4[2])
print("Elementos da 4to lista =", lista4[3])
print()
print("Elementos da 1ra lista =", lista4[0])
print("Acessando ao 6to elemento da 1ra lista =", lista4[0][5])
```

```
print("Tamanho a lista 4 = ", len(lista4))  
print("Tamanho a lista 4 = ", len(lista4[0]))
```

Figura E: exemplo74.py

Exemplo 75

PASTA: IPE-Python-2023/ Aula07-Python

arquivo: exemplo75.py

```
lista_numeros = [70, 10, 40, 30, 20, 80, 50, 90]  
print("Lista original lista_numeros = ", lista_numeros)  
print("Novo tamanho da lista de caracteres =", len(lista_numeros))  
  
lista2 = 4 * [0]  
print("Lista 2 = ", lista2)  
  
print("Inclui um elemento no final da lista de numeros")  
lista_numeros.append(100)  
print("Nova lista de numeros =", lista_numeros)  
print()  
print("Novo tamanho da lista de caracteres =", len(lista_numeros))  
  
lista2.insert(2,880)  
print("Nova lista de caracteres =", lista2)  
print("Novo tamanho da lista de caracteres =", len(lista2))  
lista2.insert(1,555)  
print("Nova lista de caracteres =", lista2)  
print("Novo tamanho da lista de caracteres =", len(lista2))  
  
lista_numeros.remove(90)  
print("Nova lista de caracteres =", lista_numeros)  
lista_numeros.append(100)  
lista_numeros.append(100)  
print("Retorna o numero de ocorrencia do elemento 100 =", lista_numeros.count(100))  
print("Retorna a posição do elemento 880 = ", lista2.index(880))  
  
print("Elementos da lista2 = ", lista2)  
print("tamanho da lista 2 = ", len(lista2))  
  
lista2.clear()  
print("Elementos da lista2 = ", lista2)  
  
print("Retorna a lista original dos numeros = ", lista_numeros)  
print("Lista em ordem crescente", sorted(lista_numeros))
```

```
lista_numeros.reverse()
print("Retorna a lista invertida = ", lista_numeros)
print("Lista em ordem crescente", sorted(lista_numeros))
```

Figura F: exemplo75.py

Exemplo 76

PASTA: IPE-Python-2023/ Aula07-Python

arquivo: exemplo76.py

```
lista_caracteres = ["a", "b", "c", "d", "e", "f", "g"]
lista_numeros = [700, 100, 400, 300, 700, 600, 800, 500]
lista5 = [9, 4, 8, 7, 3]

print("Elementos da lista de caracteres =", lista_caracteres)
print("Tamanho da lista de caracteres =", len(lista_caracteres))
print("Incluir um elemento no final da lista de caracteres ")
lista_caracteres.append("t")
print("Nova lista de caracteres =", lista_caracteres)
print()
print("Elementos da lista de caracteres =", lista_numeros)
print("Novo tamanho da lista de caracteres =", len(lista_numeros))
print("Incluir um elemento no final da lista de numeros")
lista_numeros.append(6500)
print("Nova lista de caracteres =", lista_numeros)
print("Novo tamanho da lista de caracteres =", len(lista_numeros))
print()

lista_numeros.insert(2,8000)
print("Nova lista de caracteres =", lista_numeros)
print("Novo tamanho da lista de caracteres =", len(lista_numeros))

lista_numeros.remove(400)
print("Nova lista de caracteres =", lista_numeros)

print(" Retorna o numero de ocorrencia do elemento 100 =", lista_numeros.count(100))
print("Retorna a posição do elemento 700 = ", lista_numeros.index(700))

print("Elementos da lista5 = ", lista5)
print("tamanho da lista 5 = ", len(lista5))
lista5.clear()
print("Elementos da lista5_vazia = ", lista5)

lista_numeros.reverse()
print("Retorna a lista invertida = ", lista_numeros)
print("Lista em ordem crescente", sorted(lista_numeros))

print("listagem dos elementos da lista numeros", lista_numeros)
```

```
lista_numeros.pop()
print("Nova lista com a eliminação do ultimo elemento", lista_numeros)

print("Numero minimo da Lista = ", min(lista_caracteres))
print("Numero maximo da Lista = ", max(lista_caracteres))

print("Numero minimo da Lista = ", min(lista_numeros))
print("Numero maximo da Lista = ", max(lista_numeros))
print("Soma de todos os elementos da Lista = ",sum(lista_numeros))
```

Figura G: exemplo76.py